

LibeRties reference manual

Portable Job Execution for LibeR

Version 0.7.0

Generated 2026-07-22

30 help topics

Contents

- LibeRQueue
- LibeRRemote
- LibeRServer
- ls_admin_gui
- ls_api
- ls_audit_read
- ls_generate_storage_key
- ls_job
- ls_job_decode
- ls_job_encode
- ls_job_from_wire
- ls_job_manifest
- ls_job_to_wire
- ls_library_job
- ls_local_queue
- ls_queue_capabilities
- ls_remote
- ls_result_decode
- ls_result_from_wire
- ls_result_to_wire
- ls_run_admin
- ls_run_api
- ls_security_policy
- ls_server
- ls_server_preflight
- ls_user_create
- ls_user_delete
- ls_user_list
- ls_user_rotate_token
- ls_user_update

Persistent cross-platform LibeR job queue

Description

Each user receives a separate filesystem namespace. Live jobs run in fresh 'callr' R processes and reconstruct C++ pointers from serialized models.

Public fields

`root`

Normalized persistent queue storage directory.

`user`

Safe tenant namespace used by default for queue operations.

`max_workers`

Maximum simultaneous background workers.

`limits`

Effective resource and storage limits.

`isolation`

Description of the worker isolation strategy.

`processes`

Internal environment of live 'callr' worker handles.

Methods

Public methods

- `LibeRQueue$new()`
- `LibeRQueue$submit()`
- `LibeRQueue$poll()`
- `LibeRQueue$status()`

- `LibeRQueue$list()`
 - `LibeRQueue$result()`
 - `LibeRQueue$logs()`
 - `LibeRQueue$cancel()`
 - `LibeRQueue$wait()`
 - `LibeRQueue$print()`
 - `LibeRQueue$clone()`
-

`LibeRQueue$new()`

Create or reopen a persistent local queue.

Usage

```
LibeRQueue$new(  
  root = .ls_default_root(),  
  user = "local",  
  max_workers = 1L,  
  limits = list()  
)
```

Arguments

root

Persistent queue storage directory.

user

Default isolated tenant namespace.

max_workers

Maximum simultaneous background workers.

limits

Named overrides for runtime, CPU, memory, payload, result, concurrency, queue, and storage limits.

Returns

A new 'LibeRQueue' object.

LibeRQueue\$submit()

Persist a serializable job and optionally start available workers.

Usage

```
LibeRQueue$submit(job, user = self$user, start = TRUE)
```

Arguments

job

A job created by [ls_job()].

user

Tenant namespace receiving the job.

start

Start available queued work immediately.

Returns

The durable job identifier, invisibly.

LibeRQueue\$poll()

Refresh worker state, enforce limits, recover jobs, and start queued work.

Usage

```
LibeRQueue$poll(start = TRUE)
```

Arguments

start

Whether to start queued work when capacity is available.

Returns

The current job table, invisibly.

LibeQueue\$status()

Read durable metadata for one job.

Usage

```
LibeQueue$status(id, user = self$user)
```

Arguments

id

Durable job identifier.

user

Tenant namespace owning the job.

Returns

A named metadata list.

LibeQueue\$list()

List durable jobs in a tenant namespace.

Usage

```
LibeQueue$list(user = self$user)
```

Arguments

user

Tenant namespace to list.

Returns

A data frame ordered from newest to oldest submission.

LibeRQueue\$result()

Read and verify a completed job result.

Usage

```
LibeRQueue$result(id, user = self$user)
```

Arguments

id

Durable job identifier.

user

Tenant namespace owning the job.

Returns

The deserialized job result.

LibeRQueue\$logs()

Read a worker log stream.

Usage

```
LibeRQueue$logs(id, user = self$user, stream = c("stdout", "stderr"))
```

Arguments

id

Durable job identifier.

user

Tenant namespace owning the job.

stream

Standard-output or standard-error stream.

Returns

A character vector containing log lines.

LibeRQueue\$cancel()

Cancel a queued or running job.

Usage

```
LibeRQueue$cancel(id, user = self$user)
```

Arguments

id

Durable job identifier.

user

Tenant namespace owning the job.

Returns

'TRUE' when cancellation changed the job state and 'FALSE' when the job was already terminal, invisibly.

LibeRQueue\$wait()

Poll until a job reaches a terminal state.

Usage

```
LibeRQueue$wait(id, user = self$user, timeout = Inf, poll_interval = 0.1)
```

Arguments

id

Durable job identifier.

user

Tenant namespace owning the job.

timeout

Maximum elapsed seconds; 'Inf' waits indefinitely.

poll_interval

Delay between polls in seconds.

Returns

Final job metadata.

LibeRQueue\$print()

Print queue location, capacity, limits, and job count.

Usage

```
LibeRQueue$print(...)
```

Arguments

...

Unused.

Returns

The queue, invisibly.

LibeRQueue\$clone()

The objects of this class are cloneable with this method.

Usage

```
LibeRQueue$clone(deep = FALSE)
```

Arguments

deep

Whether to make a deep clone.

LibeRties remote-server client

Description

LibeRties remote-server client

Public fields

`url`

Normalized HTTP(S) service base URL.

`token`

Bearer token used for authenticated requests.

`timeout`

Per-request timeout in seconds.

Methods

Public methods

- `LibeRRemote$new()`
- `LibeRRemote$authenticate()`
- `LibeRRemote$submit()`
- `LibeRRemote$list()`
- `LibeRRemote$status()`
- `LibeRRemote$result()`
- `LibeRRemote$logs()`
- `LibeRRemote$cancel()`
- `LibeRRemote$clone()`

`LibeRRemote$new()`

Configure an authenticated remote client.

Usage

```
LibeRRemote$new(url, token, timeout = 60)
```

Arguments

`url`

HTTP(S) service base URL.

`token`

Bearer token issued by [`ls_user_create()`].

`timeout`

Per-request timeout in seconds.

Returns

A new 'LibeRRemote' object.

LibeRRemote\$authenticate()

Verify the bearer token and return the authenticated user metadata.

Usage

```
LibeRRemote$authenticate()
```

Returns

Server authentication metadata.

LibeRRemote\$submit()

Submit a typed, non-executable job payload.

Usage

```
LibeRRemote$submit(job)
```

Arguments

job

A job created by [ls_job()].

Returns

The durable remote job identifier.

LibeRRemote\$list()

List jobs belonging to the authenticated user.

Usage

```
LibeRRemote$list()
```

Returns

A data frame of remote jobs.

LibeRRemote\$status()

Read the status and metadata of one remote job.

Usage

```
LibeRRemote$status(id)
```

Arguments

id

Durable remote job identifier.

Returns

A named metadata list.

LibeRRemote\$result()

Download and validate a completed result.

Usage

```
LibeRRemote$result(id)
```

Arguments

id

Durable remote job identifier.

Returns

The reconstructed result object.

LibeRRemote\$logs()

Download a remote worker log stream.

Usage

```
LibeRRemote$logs(id, stream = c("stdout", "stderr"))
```

Arguments

id

Durable remote job identifier.

stream

Standard-output or standard-error stream.

Returns

A character vector containing log lines.

LibeRRemote\$cancel()

Request cancellation of a queued or running remote job.

Usage

```
LibeRRemote$cancel(id)
```

Arguments

`id`

Durable remote job identifier.

Returns

'TRUE' if the server accepted cancellation.

LibeRRemote\$clone()

The objects of this class are cloneable with this method.

Usage

```
LibeRRemote$clone(deep = FALSE)
```

Arguments

`deep`

Whether to make a deep clone.

Authenticated multi-tenant LibeRties server core

Description

This object is the transport-independent server boundary used by a local test host and, later, the HTTP adapter. Every job operation derives the user namespace from the bearer token; callers cannot nominate another tenant.

Public fields

`root`

Normalized persistent server storage directory.

`max_workers_per_user`

Host-level simultaneous-worker ceiling per tenant.

Methods

Public methods

- `LibeRServer$new()`
- `LibeRServer$authenticate()`
- `LibeRServer$submit()`
- `LibeRServer$poll()`
- `LibeRServer$list()`
- `LibeRServer$status()`
- `LibeRServer$result()`
- `LibeRServer$logs()`
- `LibeRServer$cancel()`
- `LibeRServer$clone()`

`LibeRServer$new()`

Create or reopen the transport-independent server core.

Usage

```
LibeRServer$new(root = .ls_default_root(), max_workers_per_user = 2L)
```

Arguments

root

Persistent server storage directory.

max_workers_per_user

Host-level simultaneous-worker ceiling per tenant.

Returns

A new 'LibeRServer' object.

LibeRServer\$authenticate()

Authenticate a bearer token without exposing credential material.

Usage

```
LibeRServer$authenticate(token, scope = NULL)
```

Arguments

token

Bearer token issued by [ls_user_create()].

scope

Optional required API permission.

Returns

User metadata and effective limits.

LibeRServer\$submit()

Submit a job to the authenticated user's isolated queue.

Usage

```
LibeRServer$submit(token, job, start = TRUE)
```

Arguments

token

Bearer token issued by [ls_user_create()].

job

A job created by [ls_job()].

start

Start available work immediately.

Returns

The durable job identifier, invisibly.

LibeRServer\$poll()

Poll the authenticated user's queue.

Usage

```
LibeRServer$poll(token, start = TRUE)
```

Arguments

token

Bearer token issued by [ls_user_create()].

start

Start available work immediately.

Returns

The current job table, invisibly.

LibeRServer\$list()

List jobs for the authenticated user.

Usage

```
LibeRServer$list(token)
```

Arguments

token

Bearer token issued by [ls_user_create()].

Returns

A data frame of durable jobs.

LibeRServer\$status()

Read metadata for one authenticated-user job.

Usage

```
LibeRServer$status(token, id)
```

Arguments

token

Bearer token issued by [ls_user_create()].

id

Durable job identifier.

Returns

A named metadata list.

LibeRServer\$result()

Read and size-check one completed result.

Usage

```
LibeRServer$result(token, id)
```

Arguments

token

Bearer token issued by [ls_user_create()].

id

Durable job identifier.

Returns

The deserialized result object.

LibeRServer\$logs()

Read a worker log for an authenticated-user job.

Usage

```
LibeRServer$logs(token, id, stream = c("stdout", "stderr"))
```

Arguments

token

Bearer token issued by [ls_user_create()].

id

Durable job identifier.

stream

Standard-output or standard-error stream.

Returns

A character vector containing log lines.

LibeRServer\$cancel()

Cancel an authenticated-user job.

Usage

```
LibeRServer$cancel(token, id)
```

Arguments

token

Bearer token issued by [ls_user_create()].

id

Durable job identifier.

Returns

'TRUE' when cancellation changed the job state and 'FALSE' when the job was already terminal, invisibly.

LibeRServer\$clone()

The objects of this class are cloneable with this method.

Usage

```
LibeRServer$clone(deep = FALSE)
```

Arguments

deep

Whether to make a deep clone.

Create the LibeRties server-administration application

Description

The browser session is protected by an administrator token whose digest is held only in the app process. It is deliberately separate from tenant API tokens. Use a long random value supplied through 'LIBERTIES_ADMIN_TOKEN'.

Usage

```
ls_admin_gui(  
  root = .ls_default_root(),  
  admin_token = Sys.getenv("LIBERTIES_ADMIN_TOKEN")  
)
```

Arguments

root	LibeRties server storage root.
------	--------------------------------

admin_token	Administrator login token. Defaults to the 'LIBERTIES_ADMIN_TOKEN' environment variable.
-------------	--

Value

A Shiny application object.

Build the authenticated LibeRties HTTP API

Description

The API accepts only the typed JSON wire contract. It does not expose an RDS deserializer or an endpoint capable of evaluating submitted R code.

Usage

```
ls_api(server = ls_server(), policy = ls_security_policy())
```

Arguments

server	A [LibeRServer] instance.
--------	---------------------------

policy	HTTP security and rate-limit policy.
--------	--------------------------------------

Value

A configured Plumber router.

Read and verify the LibeRties administration audit chain

Description

Read and verify the LibeRties administration audit chain

Usage

```
ls_audit_read(root = .ls_default_root())
```

Arguments

root	LibeRties server root.
------	------------------------

Value

A data frame with a logical 'valid' attribute.

Generate a LibeRties storage-encryption key

Description

The returned 64-character secret is shown once and should be placed in a managed secret store as 'LIBERTIES_STORAGE_KEY', never in source control.

Usage

```
ls_generate_storage_key()
```

Create a versioned LibeR execution job

Description

Create a versioned LibeR execution job

Usage

```
ls_job(  
  type = c("simulate", "estimate", "estimate_sequence", "individualise",  
"regimen",  
    "optimal_design"),  
  model,  
  data,  
  arguments = list(),  
  label = NULL  
)
```

Arguments

type	'simulate', 'estimate', 'estimate_sequence', 'individualise', 'regimen', or 'optimal_design'.
------	---

model	A serializable LibeRation model (never a live external pointer).
-------	--

data	A serializable NONMEM-style dataset.
------	--------------------------------------

arguments	Named arguments passed to the selected LibeRation entry point.
-----------	--

label	Optional human-readable label.
-------	--------------------------------

Value

A serializable 'liber_job'.

Decode and validate a LibeR JSON job

Description

Decode and validate a LibeR JSON job

Usage

```
ls_job_decode(json, max_bytes = 100 * 1024^2)
```

Arguments

json	UTF-8 JSON text or raw bytes.
------	-------------------------------

max_bytes	Maximum accepted encoded payload size.
-----------	--

Encode a LibeR job as JSON

Description

Encode a LibeR job as JSON

Usage

```
ls_job_encode(job)
```

Arguments

job	A [ls_job()] object.
-----	----------------------

Rebuild and validate a LibeR job from the JSON wire contract

Description

The transmitted expression IR is never trusted. The semantic model fields are compiled again through [LibeRation::nm_model()] on the receiving host.

Usage

```
ls_job_from_wire(payload)
```

Arguments

payload	Parsed JSON-compatible payload.
---------	---------------------------------

Value

A validated 'liber_job'.

Create the transport manifest for a job payload

Description

Create the transport manifest for a job payload

Usage

```
ls_job_manifest(job)
```

Arguments

job	A [ls_job()] object.
-----	----------------------

Value

A JSON-compatible manifest with an exact serialized-payload checksum.

Convert a LibeR job to the non-executable JSON wire contract

Description

Convert a LibeR job to the non-executable JSON wire contract

Usage

```
ls_job_to_wire(job)
```

Arguments

job A [`ls_job()`] object containing an 'nm_model'.

Value

A JSON-compatible typed object.

Create a typed LibeRary literature job

Description

Create a typed LibeRary literature job

Usage

```
ls_library_job(  
  type = c("library_triage", "library_parse", "library_index",  
    "library_dual_extract",  
    "library_assess", "library_adjudicate"),  
  payload,  
  arguments = list(),  
  label = NULL  
)
```

Arguments

type	A typed LibeRary pipeline task.
payload	A data-only LibeRary payload.
arguments	Named worker controls, including sanitized provider config.
label	Optional label.

Value

A serializable 'liber_job'.

Create a local LibeR queue

Description

Create a local LibeR queue

Usage

```
ls_local_queue(  
  root = .ls_default_root(),  
  user = "local",  
  max_workers = 1L,  
  limits = list()  
)
```

Arguments

root	Persistent queue root.
user	Isolated user namespace.
max_workers	Maximum simultaneous worker subprocesses for this queue.
limits	Named resource limits, including wall time, CPU time, memory, payload, result, and storage quotas.

Value

A persistent 'LibeRQueue' object.

Examples

```
queue <- ls_local_queue(tempfile("liberties-queue-"), max_workers = 1L)  
queue$list()
```

Report queue and remote-worker contract capabilities

Description

Report queue and remote-worker contract capabilities

Usage

```
ls_queue_capabilities()
```

Connect to a LibeRties remote server

Description

Connect to a LibeRties remote server

Usage

```
ls_remote(url, token, timeout = 60)
```

Arguments

url	Server base URL.
token	Bearer token issued by [ls_user_create()].
timeout	HTTP timeout in seconds.

Value

A configured 'LibeRRemote' client.

Examples

```
remote <- ls_remote("https://liberties.example.org", "replace-with-token")
remote$url
```

Decode a remote result object

Description

Decode a remote result object

Usage

```
ls_result_decode(json, max_bytes = 500 * 1024^2)
```

Arguments

json	UTF-8 result JSON.
max_bytes	Maximum accepted result size.

Rebuild a result from its parsed wire representation

Description

Rebuild a result from its parsed wire representation

Usage

```
ls_result_from_wire(payload)
```

Arguments

payload	Parsed result wire object.
---------	----------------------------

Encode a safe result object for remote transport

Description

Encode a safe result object for remote transport

Usage

```
ls_result_to_wire(result)

ls_result_encode(result)
```

Arguments

result	Serializable result without executable or pointer-backed values.
--------	--

Run the LibeRties server-administration application

Description

Run the LibeRties server-administration application

Usage

```
ls_run_admin(  
  root = .ls_default_root(),  
  admin_token = Sys.getenv("LIBERTIES_ADMIN_TOKEN"),  
  host = "127.0.0.1",  
  port = NULL,  
  launch.browser = getOption("shiny.launch.browser", interactive()),  
  ...  
)
```

Arguments

root	LibeRties server storage root.
admin_token	Administrator token.
host	Listening host. Loopback is the secure default.
port	Listening port; 'NULL' selects a free port.
launch.browser	Passed to [shiny::runApp()].
...	Additional arguments passed to [shiny::runApp()].

Run the LibeRties HTTP service

Description

Bind to loopback by default and place a TLS reverse proxy in front of this service for remote deployment.

Usage

```
ls_run_api(  
  root = .ls_default_root(),  
  host = "127.0.0.1",  
  port = 8000L,  
  max_workers_per_user = 2L,  
  quiet = FALSE,  
  production = !.ls_loopback(host),  
  behind_tls_proxy = FALSE,  
  policy = ls_security_policy(production = production)  
)
```

Arguments

root	Persistent server root.
host	Listen address.
port	TCP port.
max_workers_per_user	Host-level per-user worker ceiling.
quiet	Suppress Plumber startup messages.
production	Enforce production preflight. Defaults to 'TRUE' for a non-loopback host.

behind_tls_proxy

Confirm that HTTPS is terminated by a maintained reverse proxy.

policy

Optional security policy.

Define a LibeRties deployment-security policy

Description

Define a LibeRties deployment-security policy

Usage

```
ls_security_policy(  
  production = FALSE,  
  requests_per_minute = if (production) 120L else 10000L,  
  require_storage_encryption = production,  
  require_os_isolation = production  
)
```

Arguments

production	Enable production requirements.
requests_per_minute	Per-credential HTTP request ceiling.
require_storage_encryption	Require 'LIBERTIES_STORAGE_KEY'.
require_os_isolation	Require 'LIBERTIES_OS_ISOLATION' to identify the external container, service-account, cgroup, or Windows Job Object layer.

Value

A validated 'liberties_security_policy'.

Create the authenticated LibeRties server core

Description

Create the authenticated LibeRties server core

Usage

```
ls_server(root = .ls_default_root(), max_workers_per_user = 2L)
```

Arguments

root	Persistent server storage root.
max_workers_per_user	Host-level worker ceiling per tenant.

Value

A 'LibeRServer' object.

Examples

```
server <- ls_server(tempfile("liberties-server-"))
server$root
```

Check whether a LibeRties deployment satisfies its security policy

Description

Check whether a LibeRties deployment satisfies its security policy

Usage

```
ls_server_preflight(  
  root = .ls_default_root(),  
  host = "127.0.0.1",  
  behind_tls_proxy = FALSE,  
  policy = ls_security_policy(production = !.ls_loopback(host)),  
  strict = FALSE  
)
```

Arguments

root	Server storage root.
host	Intended listen address.
behind_tls_proxy	Whether a maintained TLS reverse proxy terminates HTTPS before traffic reaches LibeRties.
policy	A [ls_security_policy()] object.
strict	Throw an error for unmet requirements.

Value

A 'liberties_security_preflight' report.

Create a server user and one-time bearer token

Description

Only the SHA-256 digest of the cryptographically random 256-bit token is persisted. The returned token is the only copy available to the caller.

Usage

```
ls_user_create(  
    root,  
    username,  
    limits = list(),  
    enabled = TRUE,  
    first_name = "",  
    last_name = "",  
    scopes = c("jobs:read", "jobs:write"),  
    expires = NULL  
)
```

Arguments

root	LibeRties server storage root.
username	Tenant identifier.
limits	Named resource limits.
enabled	Whether the account may authenticate.
first_name, last_name	Optional human-readable user names used by the administration interface. Authentication continues to use 'username'.
scopes	API permissions assigned to the token.

expires

Optional future expiry date-time. 'NULL' creates a non-expiring research token; production preflight reports it.

Value

User metadata plus the one-time token.

Delete a server user

Description

Active jobs must be cancelled or allowed to finish first. Job storage is retained unless 'remove_jobs = TRUE' is explicitly requested.

Usage

```
ls_user_delete(root, username, remove_jobs = FALSE)
```

Arguments

root	LibeRties server storage root.
username	Tenant identifier.
remove_jobs	Remove completed/cancelled/failed job storage as well.

Value

'TRUE', invisibly.

List server users without credential material

Description

List server users without credential material

Usage

```
ls_user_list(root)
```

Arguments

root	LibeRties server storage root.
------	--------------------------------

Rotate a server user's bearer token

Description

Rotate a server user's bearer token

Usage

```
ls_user_rotate_token(root, username, expires = NULL)
```

Arguments

root	LibeRties server storage root.
------	--------------------------------

username	Tenant identifier.
----------	--------------------

expires	Optional replacement expiry date-time; 'NULL' retains the current expiry.
---------	---

Value

New one-time bearer token.

Update server-user status and resource limits

Description

Update server-user status and resource limits

Usage

```
ls_user_update(  
    root,  
    username,  
    enabled = NULL,  
    limits = NULL,  
    first_name = NULL,  
    last_name = NULL,  
    scopes = NULL,  
    expires = NULL  
)
```

Arguments

root	LibeRties server storage root.
username	Tenant identifier.
enabled	Optional account state.
limits	Optional named limit overrides.
first_name, last_name	Optional human-readable names. Pass an empty string to clear a value and 'NULL' to leave it unchanged.
scopes, expires	Optional token permissions and expiry.